

Oberon Build Roadmap and Architecture Reference

Verified guidance for the first technical build

June 20, 2026

Table of Contents

Verification status and important corrections

Reviewed: June 20, 2026

The central recommendation is sound: Oberon should first be built as a narrow, evidence-producing change-analysis workflow rather than as a broad platform. The uploaded product blueprint reaches the same conclusion: one complete vegetation-monitoring workflow should be proven before expansion.

The statements below were checked against current official documentation for GeoJSON, STAC, Sentinel-2 Level-2A, cloud-optimized access, Rasterio, and Clay. The roadmap itself is partly an engineering judgment, so it cannot be “proven” in the same way as a data-format specification. It is nevertheless a strong and lower-risk sequence for an idea-stage product.

Confirmed

- **A polygon is the correct first geographic input.** In product language, call it an **area of interest (AOI)**. GeoJSON supports both Polygon and MultiPolygon geometries, so the implementation should eventually accept both.
- **STAC is appropriate for discovery.** STAC standardizes how spatiotemporal assets and metadata are described and searched.
- **Sentinel-2 Level-2A is a reasonable first optical source.** It provides bottom-of-atmosphere/surface-reflectance products and quality layers suitable for an initial vegetation workflow.
- **Windowed raster reads are the correct access pattern.** Rasterio supports reading rectangular subsets instead of loading entire rasters, and COG-hosted Sentinel-2 archives are designed for cloud-native access.
- **Local quality over the AOI matters more than scene-level cloud percentage.** Scene-level metadata is only a coarse filter; the actual cloud, shadow, snow, and no-data fraction should be measured over the area being analyzed.
- **A deterministic baseline must come before an AI claim.** NDVI, task-relevant indices, pixel differences, and quality measurements provide the comparison needed to determine whether AI adds value.
- **Raw model distance is not confidence.** A similarity or embedding distance should not be presented as a probability unless it is calibrated against labeled or reviewed examples.
- **Geographic and temporal holdouts are important.** Random neighboring patches can make performance appear better than it will be on new places or seasons.

Corrections and refinements

1. **The “four planes” are a useful planning model, not a formal industry standard.** Use them to reason about ownership and build order, but do not force the codebase to mirror them as four deployed services.

2. **“Choose one observation from each period” is acceptable for the first benchmark, not necessarily for production.** A robust monitoring system should be able to create a small cloud-masked composite or select multiple observations when no single acquisition is sufficiently clean.
3. **Do not hard-code Clay assumptions throughout the system.** Current Clay documentation shows a Sentinel-2 example using a 10-band, 256 x 256 chip and sensor metadata. Put those details behind a versioned model adapter so a model update or different sensor does not infect the rest of the architecture.
4. **NBR is not a universal vegetation metric.** It is especially useful for burn and disturbance-related signals. The baseline should be selected by task and may include NDVI, NBR, NDMI, raw reflectance deltas, and contextual quality features.
5. **Python-first is an execution recommendation, not a rejection of Rust.** Rust can still become the control plane after the geospatial contracts are stable. Starting with Rust would add a language boundary before the scientific workflow has been validated.
6. **The first task should be defined more narrowly than “vegetation change.”** A practical starting claim is “material vegetation disturbance within a defined AOI,” with explicit minimum area, time-window, quality, and abstention rules.

What “accept one polygon” means

It means the system receives the boundary of the land to inspect.

- The user may draw the boundary on a map or upload it as GeoJSON.
- The outer polygon is the monitored project, parcel, forest block, or restoration site.
- Oberon searches for satellite observations overlapping that boundary.
- It reads and evaluates only the relevant image window.
- It may later return smaller **finding polygons** showing where suspected change occurred inside the input boundary.

A first request therefore means:

Analyze this one defined piece of land between these two time windows for this one type of change, and return evidence that a person can verify.

Recommended first implementation target

The first milestone should be a local Python command that takes one AOI and two date windows and produces:

1. Selected source observations and their quality measurements.
2. Prepared, aligned before-and-after arrays.
3. Cloud, shadow, snow, and no-data masks.
4. Deterministic change measurements.
5. Before, after, and overlay images.
6. A machine-readable finding and provenance manifest.
7. An explicit abstention when the evidence is inadequate.

Only after this path is repeatable should you add a durable API, queue, Rust control plane, accounts, scheduling, or a customer-facing dashboard.

The correct mental model for Oberon

Oberon is **not primarily an AI model, satellite API, or dashboard.**

It is a **decision pipeline**:

Given an area and two time periods, find suitable satellite observations, make them comparable, detect meaningful change, rank the results, and return enough evidence for a human to trust or reject the finding.

That distinction should control every architectural decision. The model is only one replaceable component inside the pipeline. The valuable system is the reliable path from **raw observations to review-ready evidence**. Your blueprint correctly identifies one complete vegetation-monitoring workflow as the first product rather than a general Earth-intelligence platform.

Your biggest technical risks are not API performance or cloud scaling. They are:

- 1. Selecting genuinely comparable observations.
- 2. Handling clouds, shadows, seasonality, missing pixels, and alignment.
- 3. Distinguishing real change from observational noise.
- 4. Producing results that a user can verify.
- 5. Showing that AI improves the outcome over simple spectral methods.

Everything else is secondary until those are proven.

The big-picture architecture

Think of Oberon as four major planes.

1. Data plane

This is where satellite data becomes analysis-ready information.

Its subsystems are:

Subsystem	Responsibility
Portfolio and geometry input	Accept polygons, project boundaries, monitoring areas, and date windows
Catalog discovery	Search STAC catalogs for candidate Sentinel-2 observations
Scene quality assessment	Determine which observations are locally usable over the requested area
Raster access	Read only the required portions of cloud-optimized imagery
Preparation	Mask invalid pixels, normalize bands, reproject, resample, crop, and align dates
Baseline analytics	Compute NDVI, NBR, raw pixel differences, valid-pixel fractions, and other deterministic measurements
AI inference	Produce embeddings, feature differences, segmentation masks, or task scores
Postprocessing	Convert pixel-level output into polygons, areas, severity metrics, and ranked findings
Artifact generation	Produce before/after images, overlays, masks, GeoJSON, and supporting measurements

This is the heart of Oberon.

Most of your early engineering time should be spent here.

2. Control plane

The control plane coordinates work. It should not contain the actual geospatial or model logic.

Its subsystems are:

Subsystem	Responsibility
API or CLI	Accept a change-analysis request
Request validation	Validate geometry, dates, task type, thresholds, and configuration
Job state machine	Track pending, running, completed, failed, rejected, or abstained states
Orchestrator	Call the data-processing stages in the correct order
Queue	Allow longer-running work to happen asynchronously
Retry and recovery	Recover from transient network, storage, or worker failures
Metadata persistence	Store jobs, scene selections, outputs, versions, and status
Scheduling	Later, rerun monitoring jobs periodically
Notifications	Later, alert users when significant changes are found

Your document recommends Rust with Axum for this layer and Python for the geospatial and ML runtime. That is a reasonable target architecture, but it is **not the right starting sequence**.

Start with a Python CLI or callable pipeline. Add the Rust control plane only after the pipeline contracts and stage boundaries are stable.

Otherwise, you will spend time building orchestration around an unproven workflow.

3. Trust and decision plane

This is what separates Oberon from a basic imagery script.

Its subsystems are:

Subsystem	Responsibility
Quality policy	Decide whether inputs are good enough for analysis
Calibration	Turn scores into meaningful thresholds or ranked priorities
Abstention	Refuse to make a finding when evidence is unreliable
Provenance	Record exactly which data, model, configuration, and code version produced a finding
Evidence bundles	Package everything required to independently inspect a result
Evaluation framework	Measure performance against labeled or reviewed examples
Human review	Allow findings to be approved, dismissed, or corrected

Subsystem	Responsibility
Feedback capture	Preserve reviewed outcomes for future calibration and training

This plane is not merely “logging.” It is part of the product.

A result without provenance and evidence should be considered incomplete, even when the model score looks convincing. The blueprint correctly requires baseline comparisons, geographic and temporal holdouts, calibration, abstention, and reviewable evidence before making an AI claim.

4. Commercial operating plane

This comes later.

Its subsystems eventually include:

- Accounts and organizations
- Authentication and authorization
- Tenant isolation
- Billing and usage limits
- Managed scheduling
- Shared review queues
- Alerts and integrations
- Model and task registries
- Customer-specific thresholds
- Audit logs
- Enterprise deployment controls

Do not build this plane during the technical validation phase.

A polished multitenant platform wrapped around unreliable change detection is not a product. It is infrastructure without validated value.

The first thing you should build

Build one **walking vertical slice**.

A walking vertical slice is the smallest complete version of the product that travels through the entire real workflow.

Your first slice should do this:

1. Accept one **area of interest (AOI)**, represented as a GeoJSON Polygon or MultiPolygon.
2. Accept one before-date window and one after-date window.
3. Search one Sentinel-2 STAC catalog.
4. Rank candidate observations using quality measured over the AOI, not only scene-level cloud metadata.
5. For the first benchmark, choose one suitable acquisition from each period. For a production workflow, allow a small cloud-masked composite when a single clean acquisition is unavailable.
6. Read only the required bands over the polygon.

7. Apply masks for invalid pixels, cloud, shadow, and snow.
8. Put both observations onto the same grid.
9. Compute basic deterministic change measurements.
10. Generate before/after images and a change overlay.
11. Produce a machine-readable result.
12. Record the source scenes, bands, processing configuration, and software version.
13. Return an abstention rather than a result when the inputs are not trustworthy.

That is the first Oberon system.

It does **not** initially need:

- Rust
- A web dashboard
- User accounts
- Kubernetes
- Billing
- Multiple sensors
- Multiple task models
- Scheduled monitoring
- Real-time processing
- A model marketplace
- An elaborate microservice architecture

The first objective is not “build the Oberon platform.”

The first objective is:

Prove that Oberon can reliably produce one reviewable vegetation-change result from one real polygon.

The correct build sequence

Phase 1: Define the task contract

Before implementing much code, define what the system is claiming to detect.

“Vegetation change” is too vague. Separate possible claims such as:

- Vegetation disturbance
- Major vegetation loss
- Burn impact
- Restoration recovery
- Seasonal vegetation variation
- Land-clearing event

Your first task should probably be **material vegetation disturbance**, because recovery is harder to define and evaluate. Recovery depends heavily on season, vegetation type, rainfall, planting dates, and project expectations.

Define:

- What counts as a positive example
- What counts as a negative example
- The minimum changed area
- The date ranges that are acceptable
- What uncertainty forces abstention
- Whether the output is classification, ranking, segmentation, or all three
- What evidence a human needs to verify the result

Without a task contract, you cannot know whether either the baseline or AI is correct.

Phase 2: Build a benchmark dataset

This should happen before your production architecture.

Collect a modest but meaningful set of known examples:

- Clear vegetation-loss examples
- No-change examples
- Seasonal-change examples
- Cloud-contaminated examples
- Shadow and haze examples
- Misaligned-image examples
- Different ecosystems and regions
- Different time intervals

Do not rely only on visually attractive examples.

A small, carefully reviewed benchmark is more valuable than hundreds of unverified samples.

Split the data geographically and temporally. Randomly splitting neighboring image patches will create unrealistically strong results because nearby patches share landscape, climate, and observation characteristics.

Phase 3: Implement the deterministic baseline

Build the non-AI pipeline first.

At minimum, test:

- NDVI difference
- NBR difference when burn or disturbance sensitivity is relevant
- NDMI or another moisture-sensitive index when the task requires it
- Per-band differences
- Valid-pixel fraction
- Changed-pixel area
- Simple thresholding
- Connected-region or polygon extraction
- Ranking by changed area and change magnitude

This gives you three things:

1. A functioning product even before AI.
2. A reference against which AI must prove its value.
3. A debugging tool when AI produces confusing results.

The AI should not be assumed to be the intelligence of the product. It has to earn its place.

Phase 4: Add AI as a parallel branch

Do not immediately replace the baseline with Clay.

Run AI and deterministic methods on the same prepared observations.

Conceptually:

```
Prepared before/after observations
|
+--> Deterministic baseline
|
+--> AI feature or task model
|
+--> Comparison and ranking layer
```

This separation matters because it lets you answer:

- Does AI identify changes missed by NDVI or NBR?
- Does it rank important locations higher?
- Does it reduce analyst review time?
- Does it introduce additional false positives?
- Is the added latency and GPU complexity justified?

Useful metrics include:

- Precision at the top K results
- Recall at a chosen review budget
- False positives per 100 monitored areas
- Analyst minutes per 100 areas
- Performance by geography
- Performance by season
- Abstention frequency
- Error rate under cloud or poor-quality conditions

Raw embedding distance is not confidence. A similarity score must not be exposed as a trustworthy probability unless it has been calibrated against labeled examples.

The document is correct that AI should be removed from the default path if a deterministic method performs equally well. Oberon can still be valuable as an evidence and workflow engine.

Phase 5: Turn the experiment into a local engine

Once the vertical slice works, refactor it into explicit stages.

Each stage should have:

- A defined input
- A defined output
- Validation rules
- Version information
- Logs and metrics
- A failure policy
- A retry policy where appropriate
- Tests
- A stored manifest

For example, scene selection should output a structured `SelectedObservation`, not merely pass an informal Python dictionary into the next function.

Important conceptual contracts include:

- Change request
- Candidate observation
- Selected observation
- Prepared observation
- Baseline result
- Model result
- Finding
- Evidence bundle
- Job record
- Review decision

The names are less important than making the boundaries explicit.

Phase 6: Make it reproducible

Package the local engine so another developer can run the same example without your help.

That means:

- A predictable command
- Pinned dependencies
- Docker packaging
- A small example dataset or public example request
- Clear configuration
- Stable artifact locations
- Documented external network calls
- Repeatable outputs
- Checksums where useful
- No hidden local setup

Your Week 8 milestone should be an external person successfully running the workflow, not merely seeing your demo.

Phase 7: Add the control plane

Only after the local workflow is credible should you add:

- API request handling
- Job creation
- Asynchronous processing
- Queueing
- Job state transitions
- Storage abstraction
- Result retrieval
- Cancellation or timeout behavior
- Retry handling

At this stage, Rust becomes more reasonable.

Keep the boundary strict:

Rust should own

- API contracts
- Authentication later
- Job state
- Orchestration
- Persistence coordination
- Request validation
- Queue interaction
- Result delivery

Python should own

- STAC interaction
- Raster reads
- Masks
- Resampling and alignment
- Spectral calculations
- Clay or PyTorch inference
- Geospatial postprocessing
- Artifact production

Do not duplicate geospatial logic between Rust and Python.

Phase 8: Add review workflow and repeated monitoring

This is where Oberon starts becoming an operational product rather than an analysis endpoint.

Add:

- Portfolios containing many polygons
- Scheduled reruns
- Ranked finding queues

- Approve, reject, and uncertain review states
- Comments or analyst notes
- Alerts
- Historical comparisons
- Feedback export
- Customer-specific thresholds

This should be driven by a real pilot workflow.

Do not guess the ideal review UI before watching a GIS analyst or consultant work.

Recommended architectural style

Start with a modular monolith, not microservices

You need separable components, but you do not need independently deployed services yet.

A sensible early structure is:

- One Python processing package
- One CLI
- One benchmark package
- One artifact-storage abstraction
- One metadata database
- Later, one thin API/control-plane service
- Later, separate CPU and GPU workers

This gives you clean boundaries without distributed-system overhead.

Premature microservices would create:

- Network contracts before domain contracts are stable
- More deployment work
- Harder debugging
- More partial failure states
- More observability requirements
- Slower iteration

Separate processes when there is a demonstrated operational reason, such as GPU workers needing different hardware or raster jobs requiring independent scaling.

Use a functional-core, imperative-shell approach

The central transformations should be as deterministic as possible.

For example:

- Given these arrays, calculate NDVI.
- Given this quality mask, return valid pixels.

- Given these model outputs, produce candidate polygons.
- Given these evidence values, calculate a ranking score.

The outer shell handles:

- STAC requests
- Network reads
- Files
- Databases
- Queues
- GPUs
- Logging

This makes the scientific logic easier to test and the operational logic easier to replace.

Treat every processing stage as idempotent

Running the same stage twice with the same inputs should not corrupt state or create conflicting results.

Use:

- Stable job identifiers
- Stage identifiers
- Input checksums
- Output checksums
- Explicit state transitions
- Immutable artifacts where practical
- Safe retries
- Separate temporary and finalized outputs

Your blueprint correctly recommends asynchronous jobs, idempotency, retries, checksums, explicit states, and separation between control, raster processing, and model inference.

Store large data and metadata separately

Use object or filesystem storage for:

- Raster windows
- PNG previews
- Masks
- Model outputs
- GeoJSON exports
- Evidence bundles
- Intermediate prepared chips

Use the database for:

- Jobs
- Geometries or references to geometries

- Scene metadata
- Processing status
- Artifact locations
- Model and configuration versions
- Scores
- Review decisions
- Errors

Do not put large raster payloads directly into the operational database.

Make provenance first-class

Every finding should answer:

- Which source scenes were used?
- When were they captured?
- Which bands were read?
- Which quality masks were applied?
- What CRS and resolution were used?
- What preprocessing version was used?
- Which model and weights were used?
- Which thresholds were used?
- Which software version generated the result?
- Which artifacts correspond to the finding?

A log line saying “model completed successfully” is not provenance.

Provenance should be queryable product data attached directly to the finding.

What you need to understand, in order

1. Earth-observation fundamentals

Learn enough to reason about:

- Sentinel-2 Level-2A products
- Spectral bands and spatial resolutions
- Surface reflectance
- Scene Classification Layer masks
- Clouds, cirrus, shadows, and snow
- Coordinate reference systems
- Reprojection and resampling
- Pixel alignment
- Temporal and seasonal variation
- NDVI and NBR
- STAC catalogs

- Cloud-optimized GeoTIFFs
- Windowed raster reads

You do not need to become a remote-sensing scientist before starting, but you do need to understand why two satellite images cannot automatically be subtracted and interpreted as true change.

2. Geospatial data engineering

Understand:

- Geometry validity
- Bounding boxes versus exact polygon masks
- Raster/vector conversion
- Tiling and chips
- Nodata handling
- Band stacking
- Resolution matching
- Caching
- Artifact formats
- Memory constraints with raster arrays

The likely working ecosystem is Python with tools such as Rasterio, GDAL, NumPy, GeoPandas, Shapely, PySTAC Client, and PyTorch. Treat exact package choices as replaceable implementation details, not product boundaries.

3. Evaluation and statistical reasoning

You need to become comfortable with:

- Ground truth
- Label uncertainty
- False positives and false negatives
- Precision and recall
- Precision at K
- Ranking evaluation
- Geographic holdouts
- Temporal holdouts
- Probability calibration
- Threshold selection
- Out-of-distribution behavior
- Abstention
- Human-review cost

This is more important to the product than sophisticated model architecture.

4. Reliable job-processing systems

After the analysis works locally, learn:

- State machines
- Queues
- Idempotency
- Retry classification
- Dead-letter handling
- Timeouts
- Cancellation
- Partial failure recovery
- Object storage
- Worker isolation
- CPU versus GPU scheduling
- Observability

Do not begin here. Learn it when the system actually has work worth orchestrating.

5. User workflow and product discovery

You need to understand how the target user currently works:

- Where polygons come from
- How often areas are reviewed
- What triggers an investigation
- Which output formats they require
- How findings enter reports
- How they record confidence
- When they order commercial imagery
- When they send a person into the field
- What a missed change costs
- What a false alarm costs

These answers should influence the product more than abstract architecture preferences.

A practical 90-day roadmap

Weeks 1–2: Task and dataset

Focus on:

- Define one disturbance task.
- Select one geographic domain.
- Collect reviewed positive and negative examples.
- Understand Sentinel-2 inputs.

- Establish evaluation criteria.
- Interview potential users in parallel.

Deliverable: a written task contract and benchmark dataset.

Weeks 3–4: Complete technical benchmark

Focus on:

- STAC discovery
- Scene selection
- Windowed COG reads
- Masking
- Alignment
- NDVI/NBR/pixel baselines
- Initial Clay comparison through a versioned model adapter; derive the required band order and preprocessing from the model's current sensor metadata rather than scattering hard-coded assumptions through the pipeline
- Before/after evidence
- Held-out evaluation

Decision gate: Does AI materially improve ranking or capture useful signals not handled by the baseline?

Do not manipulate the metric until AI appears useful. A negative result is valuable because it prevents unnecessary infrastructure.

Weeks 5–6: Reusable local engine

Focus on:

- Convert notebooks into modules
- Define domain contracts
- Add validation
- Add provenance manifests
- Make stages idempotent
- Add golden test cases
- Produce stable artifacts
- Create a CLI

Deliverable: one repeatable local change-analysis command.

Weeks 7–8: Packaging and external reproducibility

Focus on:

- Docker
- Configuration
- Error messages

- Example workflow
- Documentation
- Dependency pinning
- CPU/GPU separation where necessary
- External-user test

Decision gate: Can someone who is not you run it and understand the result?

If not, do not add more features. Fix the packaging and contracts.

Weeks 9–10: Minimal control plane

Focus on:

- Create-job endpoint
- Retrieve-job endpoint
- Explicit job states
- One durable queue
- One metadata database
- Artifact storage
- Safe retries
- Timeouts
- Basic metrics

At this point, you can decide whether a Rust API provides enough value to justify the additional language boundary.

Weeks 11–12: Pilot-shaped workflow

Focus on:

- Multiple polygons
- Ranked results
- Basic approve/reject review
- Exportable evidence
- One historical backtest
- Demonstration with a prospective user
- Measurement of analyst time saved

Commercial gate: Will anyone pay for a scoped benchmark or pilot?

Do not build a full hosted SaaS merely because people say the demo is interesting.

Testing strategy

You need several distinct test layers.

Unit tests

For deterministic operations such as:

- Index calculations
- Mask construction
- Geometry validation
- Score combination
- State transitions
- Provenance serialization

Golden-case tests

Use fixed real examples with approved outputs.

These catch changes in:

- Resampling
- Mask behavior
- Model versions
- Thresholds
- Polygon generation
- Artifact rendering

Integration tests

Test:

- STAC query to scene selection
- COG window reading
- Preparation to baseline
- Preparation to inference
- Result to artifact storage

End-to-end tests

Run one complete public example and verify:

- Job completion
- Required artifacts
- Valid provenance
- Expected quality state
- Reasonable score range

Failure tests

Explicitly test:

- No suitable scene
- Excessive cloud coverage
- Missing bands
- Invalid polygon

- Dates in the wrong order
- Network interruption
- Corrupted raster
- GPU failure
- Model version unavailable
- Seasonal mismatch
- Insufficient valid pixels

A system that only works on ideal examples is a demo, not monitoring infrastructure.

What not to build first

These are attractive distractions:

- A beautiful frontend
- Natural-language Earth queries
- Multiple satellite providers
- Sentinel-1 support
- Commercial imagery support
- A generic plugin architecture
- Kubernetes
- Complex event streaming
- Organization accounts
- Fine-grained permissions
- Billing
- Enterprise SSO
- Offline deployment
- A training platform
- A model marketplace
- Automated report generation for every possible customer
- Large-scale GPU reservation
- Multiple repositories for a one-person prototype

The repository structure in the blueprint describes a plausible later commercial boundary, not necessarily the correct first-week development structure. The product discipline is to prove one complete evidence-backed workflow before expanding into cloud, enterprise, or multiple task products.

The architectural principle to keep returning to

For every proposed subsystem, ask:

1. **Which verified user problem does this solve?**
2. **What input and output contract does it own?**
3. **Can it be tested independently?**

4. **Can it fail without corrupting other stages?**
5. **Does it need to scale separately today, or only conceptually?**
6. **What evidence tells us it is worth building now?**

The best Oberon architecture is not the one with the most sophisticated components.

It is the one that lets you rapidly answer the highest-risk question:

Can a repeatable combination of scene quality handling, deterministic signals, optional AI, and transparent evidence reduce the amount of land-monitoring work a real analyst must perform?

Until that answer is demonstrated, build a benchmark and a reliable vertical slice—not a platform.

Verification references

1. **Oberon Product Strategy, Technical Blueprint and Commercialization Plan**, supplied project document, June 2026.
2. **GeoJSON Format, RFC 7946**, IETF/RFC Editor: <https://datatracker.ietf.org/doc/html/rfc7946>
3. **SpatioTemporal Asset Catalogs (STAC)**: <https://stacspec.org/>
4. **Sentinel-2 Level-2A documentation**, Copernicus Data Space Ecosystem: <https://documentation.dataspace.copernicus.eu/APIs/SentinelHub/Data/S2L2A.html>
5. **Sentinel-2 Cloud-Optimized GeoTIFFs**, AWS Registry of Open Data: <https://registry.opendata.aws/sentinel-2-l2a-cogs/>
6. **Rasterio windowed reading and writing**: <https://rasterio.readthedocs.io/en/stable/topics/windowed-rw.html>
7. **Clay Foundation Model - Basic Use**: https://clay-foundation.github.io/model/getting-started/basic_use.html
8. **Clay Foundation Model - v1.5 specification**: <https://clay-foundation.github.io/model/release-notes/specification.html>
9. Meyer et al., **Importance of spatial predictor variable selection in machine learning applications**, 2019: <https://arxiv.org/abs/1908.07805>
10. Guo et al., **On Calibration of Modern Neural Networks**, 2017: <https://arxiv.org/abs/1706.04599>

Final build rule

Build the smallest system that can produce one repeatable, reviewable, evidence-backed change result. Then expand only when a technical or customer-validation gate justifies the next layer.